06 POINTERS AND REFERENCES • DETAILED SOLUTIONS

output solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Predict the Output & Debugging (Q11–Q26)

Solution to Q11

- **Concept Tested**: Dereference read and write (`*p = *p + 10`).
- **Dry Run**: `x` starts at `15` , `p = &x` . `*p + 10` evaluates to `15 + 10 = 25` , stored back into `*p` (`x`).
- **Guaranteed Output**:

```
TEXT
25 25
```

Solution to Q12

- **Concept Tested**: Reference assignment mutates the original referent instead of rebinding.
- **Dry Run**: `ref` binds to `a` (`5`). `ref = b;` assigns `10` into `a` (`a` is now `10` , `ref` still aliases `a`). `ref = ref + 3;` updates `a` to `13` . `b` remains `10` .
- **Guaranteed Output**:

```
TEXT
13 10
```

Solution to Q13 (`*p++` vs `(*p)++`)

- **Concept Tested**: Expression 1 (`*p++`) vs Expression 2 (`(*p)++`).
- **Step-by-Step Dry Run**:
  1. Initial: `arr = {10, 20, 30}` , `p = &arr[0]` .
  2. `cout << *p++` : Prints old `*p` (`arr[0] = 10`), advances `p` to `&arr[1]` .
  3. `cout << (*p)++` : `p` is at `&arr[1]` (`20`). Prints old `arr[1]` (`20`), increments `arr[1]` in memory to `21` . `p` stays at `&arr[1]` .
  4. Final print: `arr[0]` is `10` , `arr[1]` is `21` , `*p` is `21` .
- **Guaranteed Output**:

```
TEXT
10 20 10 21 21
```

Solution to Q14 (`++*p` vs `*++p`)

- **Concept Tested**: Expression 3 (`++*p`) vs Expression 4 (`*++p`).
- **Step-by-Step Dry Run**:
  1. Initial: `arr = {10, 20, 30}` , `p = &arr[0]` .
  2. `cout << ++*p` : Groups as `++(*p)` . Increments `arr[0]` from `10` to `11` and prints `11` . `p` stays at `&arr[0]` .
  3. `cout << *++p` : Groups as `*(++p)` . Advances `p` to `&arr[1]` and prints `arr[1]` (`20`).
  4. Final print: `arr[0]` is `11` , `arr[1]` is `20` , `*p` is `20` .
- **Guaranteed Output**:

```
TEXT
11 20 11 20 20
```

Solution to Q15 (`*p--` , `--*p` , `*--p`)

- **Concept Tested**: Expressions 7, 9, and 10.
- **Step-by-Step Dry Run**:
  1. Initial: `arr = {100, 200, 300}` , `p = &arr[2]` .
  2. `*p--` : Prints `arr[2]` (`300`), moves `p` to `&arr[1]` .
  3. `--*p` : Pre-decrements `arr[1]` from `200` to `199` , prints `199` . `p` stays at `&arr[1]` .
  4. `*--p` : Pre-decrements `p` to `&arr[0]` , prints `arr[0]` (`100`).
- **Guaranteed Output**:

```
TEXT
300 199 100 100 199 300
```

Solution to Q16 (`*(p + 1)` , `*(p - 1)` , `*p + 1`)

- **Concept Tested**: Expressions 11, 12, and 13.
- **Dry Run**: `p = &arr[1]` (`8`). `*(p + 1)` is `arr[2]` (`12`), `*(p - 1)` is `arr[0]` (`4`), `*p + 1` is `8 + 1 = 9` , and `*p` remains `8` . Note: Since C++17, `<<` chaining evaluates strictly Left-to-Right, and none of these expressions have side effects.
- **Guaranteed Output**:

```
TEXT
12 4 9 8
```

Solution to Q17 (`int**` Double Indirection)

- **Concept Tested**: Modifying `*pp` (rebinding `p`) vs `**pp` (modifying target `int`).

- **Dry Run**:
  1. `**pp = 15;` changes `x` from `10` to `15`.
  2. `*pp = &y;` changes `p` so that `p` now points to `y`.
  3. `**pp = 25;` changes `y` from `20` to `25`.
- **Guaranteed Output**:

```
TEXT
15 25 25
```

Solution to Q18 (Pointer Subtraction `q - p`)

- **Concept Tested**: Subtracting two pointers into the same array returns the number of elements (`std::ptrdiff_t`) between them, **not** the raw byte difference.
- **Dry Run**: `p = &arr[1]`, `q = &arr[4]`. `q - p = 4 - 1 = 3`. `*(p + 3/2) = *(p + 1) = arr[2] = 6`.
- **Guaranteed Output**:

```
TEXT
3 6
```

Solution to Q19 (Pointer Passed by Value vs. Reference to Pointer `int*&`)

- **Concept Tested**: `void try_reassign(int* ptr)` only mutates its local copy of the pointer, whereas `void real_reassign(int*& ptr)` mutates the caller's pointer `p`.
- **Guaranteed Output**:

```
TEXT
10 100
```

Solution to Q20 (`(*p++) * (*p++)`)

- **Concept Tested**: Unsequenced side effects (`[intro.execution]`).
- **Answer**: **No guaranteed output (Undefined Behavior).** Even with parentheses around `(*p++)`, the binary multiplication operator `*` does not sequence its left operand relative to its right operand. Modifying `p` twice without sequencing is Undefined Behavior in C++23.

Solutions to Debugging Questions (Q21–Q26)

- **Q21 Solution**: In `*ptr.display();`, the dot operator `.` has higher precedence (Level 2) than unary `*` (Level 3), so the compiler parses it as `*(ptr.display())` and errors because `ptr` is a pointer, not a class object.
  - **Fix 1**: `ptr→display();`

  - **Fix 2**: `(*ptr).display();`
- **Q22 Solution**:
  - **Line A (`*p1 = 30;`)**: Compile error because `p1` is a pointer to `const int` (`const int*`).
  - **Line B (`p2 = &y;`)**: Compile error because `p2` is a `const` pointer (`int* const`).
- **Q23 Solution**: `temp` is a local automatic (stack) variable that is destroyed when `create_number` returns. Returning `&temp` yields a **dangling pointer**, and dereferencing it causes Undefined Behavior. **Fix**: Return `temp` by value (`int create_number(int val) { return val; }`) or allocate with `std::make_unique<int>(val)`.
- **Q24 Solution**: Memory allocated with array `new[]` (`new int[50]`) **must** be deallocated with array `delete[]` (`delete[] arr;`). Using scalar `delete arr;` on an array allocation is **Undefined Behavior** (`[expr.delete]`).
- **Q25 Solution**: `void* vp` cannot be dereferenced directly because `void` is an incomplete type. **Fix**: `cout << *static_cast<double*>(vp) << endl;`.
- **Q26 Solution**: Both `p1` and `p2` hold the exact same heap address. After `delete p1;`, the heap block is freed and `p2` becomes a dangling pointer. Executing `delete p2;` performs a **double deletion (double free)**, which is Undefined Behavior.